

Warm up

Apply the right-left rule to the following declarations:

`int *p[10];`

- p is an array of length 10 of pointer to `int`

`int (*p)[10];`

- p is a pointer to an array of length 10 of `int`

`int **(*p)[10];`

- p is a pointer to an array of length 10 of pointer to pointer to `int`

`int (*p[10])();`

- p is an array of length 10 of pointer to function returning `int`

`int *(p[5][2]());`

- p is an array of length 5 of array of length 2 of pointer to function returning pointer to `int`

Precept 14: Abstract objects and Assignment 4

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

March 24, 2026

Overview

- 1 Abstract objects
- 2 Assignment 4: Directory and file trees
- 3 Summary

Recall: the stack ADT

```
/* stack.h */
typedef struct Stack *Stack_T;

/* ... declarations of stack operations ... */

/* stack.c */
struct Stack {
    double *pdArray;
    size_t uTop;
    size_t uPhysLength;
};

/* ... implementations of stack operations ... */
```

- Why? Encapsulation and data hiding via opaque pointers → modularity
 - Client cannot access or modify internal state of the stack object directly → can only interact with the stack via the provided operations

Recall: `static` and `extern`

- The `static` and `extern` keywords can be used to control the linkage and storage duration of global variables
- `static` creates an identifier with internal linkage (i.e., only accessible within the file it is declared)
- `extern` creates an identifier with external linkage (i.e., accessible across files)
 - An `extern` declaration does not create a new identifier; it just *declares* that an identifier with external linkage exists and is defined in some other file
 - An externally-linked *definition* must be provided in some file
 - Default for global variables

```
/* file1.c */  
extern int x;
```

```
/* file2.c */  
int x = 42;
```

Abstract objects

- Abstract objects are another style of modular design
- Properties of abstract objects:
 - Encapsulation and data hiding via static global variables
 - ◇ Not visible outside the implementation file → cannot be accessed or modified by client code
 - ◇ Live in the BSS or data segment (not the heap)
 - Interface exposes only functions → client code can only interact with the abstract object via the provided functions
 - Disadvantage: can only have one instance of the abstract object in the program
 - ◇ Unlike ADTs, where we can create multiple instances of the same ADT (e.g., multiple stack objects)
- Like a “singleton” design pattern in Java

The stack abstract object: interface

```
/* stackao.h */  
  
int Stack_init(void);  
  
void Stack_free(void);  
  
int Stack_push(double dItem);  
  
double Stack_pop(void);  
  
int Stack_isEmpty(void);
```

- Observe: none of the functions are associated with any particular stack object (i.e., no stack object as an argument)
- Consequence: assumes that there will be exactly one stack object in the program at any given time

The stack abstract object: implementation

```
/* stackao.c */

static double *pdArray; /* the array */

static size_t uTop; /* index one beyond top elt */

static size_t uPhysLength; /* num elts in array */

static int iInitialized = FALSE; /* is initialised? */

/* ... implementations of Stack_init, Stack_free,
   Stack_push, Stack_pop, Stack_isEmpty ... */
```

- Observe: the state of the stack object is stored in static global variables, which are only accessible within `stackao.c` (i.e., not accessible to clients of the stack abstract object)
 - Still accomplishes encapsulation and data hiding of an ADT!
- Where are the global variables stored? Process duration and uninitialised → stored in BSS segment

The stack abstract object: client code

```
/* teststackao.c */

/* memory error handling omitted for brevity */
int iSuccessful;

iSuccessful = Stack_init();
iSuccessful = Stack_push(1.1);
iSuccessful = Stack_push(2.2);
iSuccessful = Stack_push(3.3);

while (! Stack_isEmpty())
    printf("%g\n", Stack_pop());

Stack_free();
```

- Advantages? Simplicity: client code need not keep track of the stack object and pass it as an argument to stack operations
- Disadvantages? Cannot have multiple stack objects in the same program (e.g., if we wanted to maintain two separate stacks)

Modules we've seen so far

Module	Example	Parallel to Java
Stateless module	<code>intmath</code> , <code>str</code> (Assignment 2)	Class with static methods and no fields
Abstract data type	<code>stack</code> , <code>SymTable</code> (Assignment 3), <code>DynArray</code>	Class with instance methods and fields
Abstract object	<code>stackao</code> , Assignment 4 <code>trees</code>	Class with static methods and static fields

Validating the stack abstract object

- What are the invariants of the stack abstract object? i.e., what conditions must hold for the stack to be in a valid state? Invalid if:
 - `iInitialized` is 0, but `pdArray` is not NULL, or `uTop` is not 0, or `uPhysLength` is not 0
 - `iInitialized` is 1, but `pdArray` is NULL, or `uPhysLength` is 0
 - `uTop > uPhysLength`
- What are the invariants if a linked list is used instead of a dynamically resizing array? Invalid if:
 - `iInitialized` is 0, but `psFirstNode` is not NULL
 - `iInitialized` is 1 and `psFirstNode` is not NULL, but there is a cycle in the linked list
- Example of validation module in `stackaochecker.{h,c}`
- Validate on every return path from every stack operation (`stackaochecked.c`)
 - Trade-offs? Catches bugs as soon as they occur, but adds overhead to every stack operation

Overview

- 1 Abstract objects
- 2 Assignment 4: Directory and file trees
- 3 Summary

Assignment 4 logistics

- Can work with a partner (strongly recommended) or alone
 - Stay after precept if you want to find a partner
 - Or, find a partner via Ed Discussion post #116
- Use git and GitHub to collaborate with your partner
- Assignment due Tue, 4/7 at 9pm
 - Quiz on Mon, 4/13 in lecture

Data structures

- Binary Directory Tree (BDT)
 - A hierarchy of directories in which each node may have at most 2 children
 - All nodes are directories, no files
- Directory Tree (DT)
 - A generalisation of a BDT in which each directory may have an arbitrary number of children
 - Will discuss in next precept
 - Still, all nodes are directories, no files
- File Tree (FT)
 - An extension of a DT in which now leaves may represent either an empty directory or a file

Assignment 4 has three parts

- Part 1: Binary Directory Tree (BDT)

- Use `gdb`, `meminfo` and `valgrind` to debug and fix memory errors in the provided implementation
- Provided: object files for buggy implementations and for a correct implementation, and client source code
 - ◇ Object files were built with `-g` flag

- Part 2: Directory Tree (DT)

- Write internal validation module to verify validity of the data structure
- Provided: object files for buggy implementations, and source code for a correct implementation and simple test client
 - ◇ Object files not built with `-g` flag → debugging is less straightforward

- Part 3: File Tree (FT)

- Implement the file tree interface
- You get to design the relevant data structures and algorithms
 - ◇ Can use provided implementation of DT as a starting point, but will need to modify it to accommodate files

Overview

- 1 Abstract objects
- 2 Assignment 4: Directory and file trees
- 3 Summary

Summary

- Abstract objects, and how they differ from ADTs
- Validating abstract objects via invariants and a validation module
- Introduction to Assignment 4
- Dates to remember:
 - Assignment 3: due Tue, 3/24 at 9pm (tonight!)
 - Assignment 3 Quiz: Wed, 3/25 in lecture (tomorrow!)
 - Recommended: find an Assignment 4 partner asap